

Test Coverage & Quality Assurance Guide

Doc Ref: ACQO-DOC-005 | v1.0 | 2025

Autonomous Code Quality & Optimization Intelligence System

1. Overview

Test coverage and quality assurance are foundational to maintaining reliable, high-quality software. This guide defines unit and integration testing standards, coverage measurement thresholds, mocking principles, and CI testing requirements for all repositories monitored under the ACQO system.

2. Current Test Coverage Baseline

Repository	Key File	Coverage (%)	SLO Target	Gap	Risk Level
payment-service	PaymentProcessor.java	81.2%	85%	-3.8pp	Medium
analytics-engine	data_pipeline.py	85.6%	85%	+0.6pp	Low
user-auth-service	AuthManager.java	74.4%	90%	-15.6pp	High
reporting-dashboard	dashboardController.js	88.9%	80%	+8.9pp	Low

■ **Coverage Alert:** AuthManager.java (user-auth-service) is 15.6pp below the 90% SLO. Given its active security vulnerability, low test coverage compounds deployment risk significantly.

3. Unit Testing Principles

- **Arrange-Act-Assert (AAA):** Every unit test must follow the three-phase structure: set up test data, execute the unit under test, then assert outcomes.
- **Single Assertion Focus:** Each test case should validate one specific behaviour. Multiple assertion failures obscure root cause.
- **Test Naming Convention:** Use the pattern *methodName_givenCondition_expectedBehaviour* for clarity in test reports.
- **No Business Logic in Tests:** Tests must not contain conditional branches or loops. Complex test logic is a sign the production code is too complex.
- **Fast Execution:** Unit tests must complete in under 100ms per test. Slow tests indicate external dependency leakage.
- **Deterministic:** Tests must produce the same result regardless of execution order or system state (no clock, random, or environment dependencies without mocking).

4. Integration Testing Strategy

Test Type	Scope	Tools	Environment	Frequency
API Integration	Service endpoint contracts	REST-assured, Postman	Docker Compose	Every PR
Database Integration	Repository layer + real DB	Testcontainers (Postgres)	Isolated container	Every PR

Test Type	Scope	Tools	Environment	Frequency
Message Queue	Kafka produce/consume contracts	Embedded Kafka	In-process	Every PR
Cross-service contract	Consumer-driven contract tests	Pact	CI pipeline	Nightly
End-to-end (E2E)	Full user journey	Playwright, Selenium	Staging env	Pre-release

5. Test Coverage Measurement

Coverage is measured along four dimensions. Line coverage alone is insufficient:

Coverage Type	Definition	Minimum Threshold	Tool
Line Coverage	Percentage of lines executed by tests	80%	JaCoCo / Coverage.py
Branch Coverage	Percentage of decision branches exercised	75%	JaCoCo / Coverage.py
Mutation Coverage	Percentage of code mutations caught by tests	70%	PITest / mutmut
Path Coverage	Unique execution paths through critical functions	60%	Manual + tooling

6. Mocking & Test Isolation

- **Mock External Dependencies Only:** Mock databases, HTTP clients, and message queues in unit tests. Never mock the system under test itself.
- **Avoid Over-Mocking:** Excessive mocks produce brittle tests that pass even when production code is broken. Use real implementations in integration tests.
- **Mock Frameworks:** Use Mockito (Java), unittest.mock (Python), or Jest mocks (JavaScript). Avoid manual mock classes unless framework support is absent.
- **Testcontainers:** Use Testcontainers for database and messaging integration tests to avoid environment-specific failures in CI.

7. Continuous Integration Testing Requirements

CI Gate	Condition	Failure Behaviour	Override Allowed?
Unit Test Pass	All unit tests green	Block merge	No
Coverage Threshold	Coverage >= SLO target	Block merge	Tech Lead + justification
Security Scan	security_vulnerability_count = 0	Block merge	No
Complexity Gate	No file with cyclomatic complexity > 20	Block merge	Architecture review only
Integration Test Pass	All integration tests green	Block merge	No
Mutation Score	Mutation coverage >= 70%	Warning only	Yes — auto-escalate if <50%

CI Testing Pipeline

Code Push		Unit Tests		Coverage Gate		Integration Tests		Security Gate		Merge Allowed
✓ Recommended Practices					✗ Prohibited Practices					
✓ Write tests before refactoring to establish a safety net					✗ Merge to production with coverage below SLO without documented justification					
✓ Track coverage trends over time — regressions are early warnings					✗ Write tests that only assert the happy path in security-critical modules					
✓ Use mutation testing to validate test suite quality, not just coverage					✗ Disable integration tests to speed up the CI pipeline					